07

SKT K-뉴딜 아카데미 · AI·자동화 기초

AI 에이전트 도구 호출

LLM(Large Language Model · 엘엘엠) 이 어느 도구를 쓸지 고르고, 우리 파이썬 코드가 그 도구를 실행합니다.
위험한 도구 앞에는 사람의 승인을 둡니다.

오후에는 세 교시에 걸쳐 에이전트 하나를 만듭니다

LLM 이 고르고, 우리 코드가 실행하고, 사람이 승인하고, 파일에 남깁니다.

고르기 **LLM** → 실행 **우리 코드** → 승인 **사람** → 기록 **파일**

교시	만드는 부분	산출물	쓰는 것
5교시	**LLM 이 도구를 고르고** 우리 코드가 실행함	—	`llm_client.py`
6교시	도구 이름으로 함수를 찾아 실행하는 라우터	`tool_router.py`	**5교시의 도구 둘**
7교시	위험한 도구 앞에 사람의 승인을 두고 한 일을 남김	`agent_result.json`	**6교시의 라우터**
8교시	오늘 만든 파일 셋을 확인하고 남은 도전을 풂	—	—

5교시에 사용하는 용어는 네 개입니다

용어 (읽는 법)	뜻	5교시의 예
에이전트 (agent)	목표를 받아 스스로 순서를 정하고 도구를 쓰는 프로그램	목표 → 도구 고르기 → 실행 → 답
도구 (tool)	LLM 이 고를 수 있게 우리가 만들어 둔 함수	`count_failed_logins(args)`
도구 호출 (tool calling)	모델이 어떤 함수를 어떤 값으로 부를지 정하는 구조	`{"tool": "count_failed_logins", "args": {"user": "admin"}}`
도구 목록	쓸 수 있는 함수의 이름 · 설명 · 인자를 적은 글	`tools_text`

LLM 은 고르기만 하고 실행은 우리 코드가 합니다

오전의 LLM 은 로그를 설명하기만 했습니다.
2교시에 봤듯이 LLM 은 조회하지 않고 지어낼 수 있습니다(환각). 사실은 우리 코드가 확인합니다.

	LLM	우리 파이썬 코드
하는 일	어느 도구를 쓸지 고름	고른 도구(함수)를 실행함
211.45.12.9 의 통신사	2교시: LG Uplus · SK텔레콤 (지어냄)	`lookup_ip → South Korea / SamsungSDS Inc.`
로그 파일	열지 못함	`normalized_logs.json` 을 열어 셈

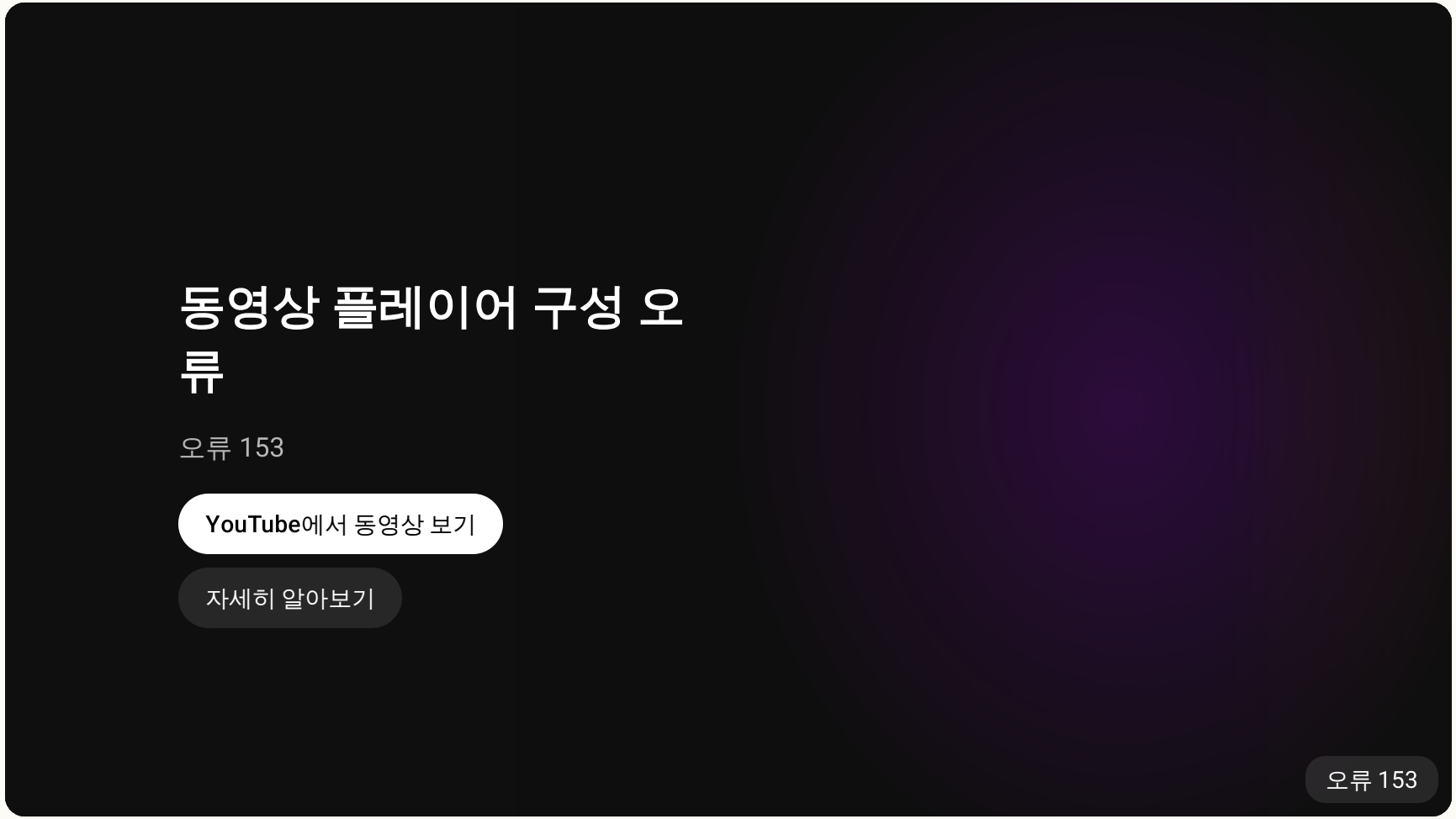
5교시에는 고르고 실행하는 흐름을 만듭니다

LLM 이 도구를 고르고, 우리 코드가 실행하고, 그 결과로 LLM 이 답합니다.

① 도구 만들기 함수 → ② 고르기 LLM → **③ 실행 우리 코드** → ④ 답 LLM 이 문장으로

순서	만드는 것	노트북	끝나면 보이는 값
① 도구	로그인 실패를 세는 함수	문제 5-2	`4`
② 고르기	도구 목록과 목표를 보내 LLM 이 고르게 함	문제 5-3	`{"tool": …, "args": …}`
③ 실행	고른 도구를 우리 코드가 실행함	문제 5-4	`South Korea / SamsungSDS Inc.`
④ 답	실행 결과를 다시 LLM 에 넘김	문제 5-5	「kim.cs 계정의 로그인 실패 횟수는 1번입니다.」

영상으로 AI 에이전트의 도구 호출을 봅니다 (6분 22초)



도구는 딕셔너리 하나를 받는 함수입니다

> **LLM 이 인자를 JSON(제이슨) 으로 주므로, 도구는 인자를 딕셔너리 하나(args)로 받습니다.**

```
import json
def count_failed_logins(args):
    with open("normalized_logs.json", encoding="utf-8") as f:
        rows = json.load(f)
    count = 0
    for row in rows:
        if row["user"] == args["user"] and row["level"] == "WARN":
            count = count + 1
    return count  # 출력하지 않고 반환
print(count_failed_logins({"user": "kim.cs"}))  # 1
```

LLM 은 우리 함수를 볼 수 없어서 도구 목록 `tools_text` 에 글로 적어 알려 줍니다

우리가 적은 다섯 문장을 + 로 이어 붙여 LLM 에게 보냅니다.
LLM 은 읽고 도구 하나를 골라 JSON 한 줄로 답할 뿐입니다.

우리가 적어 이어 붙인 문장	하는 일
「쓸 수 있는 도구는 둘입니다.」	도구가 몇 개인지 알림
「**1) count_failed_logins: 한 계정의 로그인 실패 횟수를 센다. 인자는 user.**」	도구 ①의 이름 · 하는 일 · 인자 이름 = 앞 장 함수 이름
「**2) lookup_ip: IP(아이피) 주소의 나라와 통신사를 조회한다. 인자는 ip.**」	도구 ②의 이름 · 하는 일 · 인자
「목표를 이루는 데 쓸 도구 하나를 골라 **JSON** 으로만 답하세요.」	**3교시 출력 형식 — JSON**
`예시: {"tool": "count_failed_logins", "args": {"user": "kim01"}}`	**3교시 few-shot(퓨샷) — 키 이름을 정함**

LLM 은 고른 도구를 JSON 으로 답하기만 합니다

로그 파일을 열지도, 함수를 실행하지도 않았습니다.

```
goal = "admin 계정이 로그인에 몇 번 실패했는지 알려 주세요."
text = call_llm(tools_text + "목표: " + goal)
print(text)
```

```
```json
{"tool": "count_failed_logins", "args": {"user": "admin"}}
```
```

"tool"	LLM 이 고른 도구 이름
"args"	그 도구에 넣을 인자

되풀이되는 줄을 choose_tool 함수로 묶습니다

> 「질문을 만들어 보내고 → 답을 딕셔너리로 바꾸는」 줄은 오후 내내 되풀이됩니다.

```
def choose_tool(goal):
    text = call_llm(tools_text + "목표: " + goal)
    if text == "":
        print("LLM 의 답을 받지 못했습니다. 1분 기다린 뒤 이 셀을 다시 실행합니다")
    return parse_llm_json(text)
```

`if text == "":`	답을 받지 못하면 안내를 출력함. 429 한도를 넘으면 이 경우
`return parse_llm_json(text)`	딕셔너리를 반환함. 읽지 못하면 None
쓰는 법	`choice = choose_tool(goal)` 한 줄. 문제 5-3 의 네 줄이 한 줄

없는 일을 시키면
LLM 은 있는 도구를 억지로 고릅니다

목표에 맞는 도구가 목록에 있으면 그 도구를 고릅니다.

> 그래서 6 · 7교시에 LLM 이 고른 것을 그대로 실행하지 않고 확인하는 장치를 만듭니다.

목표	LLM 이 고른 것
admin 계정이 로그인에 몇 번 실패했는지	`count_failed_logins` · `{"user": "admin"}`
185.220.101.34 는 어느 나라의 주소인지	`lookup_ip` · `{"ip": "185.220.101.34"}`
admin 계정을 잠가 주세요	`count_failed_logins` — 잠그는 도구가 없어 억지로 고름
오늘 점심 메뉴를 추천해 주세요	「관련된 도구가 없습니다」 문장 → None

LLM 은 이름만 고르므로
실행은 우리 코드가 if · elif 로 합니다

```
choice = choose_tool("admin 계정이 로그인에 몇 번 실패했는지 알려 주세요.")
if choice["tool"] == "count_failed_logins":
    result = count_failed_logins(choice["args"])
elif choice["tool"] == "lookup_ip":
    result = lookup_ip(choice["args"])
print("실행 결과:", result)
```

```
실행 결과: 4
```

choice["tool"]	LLM 이 고른 이름으로 갈래를 정함
choice["args"]	고른 인자를 그대로 도구에 넣음

실행 결과를 다시 LLM 에 넘겨 한 문장 답을 만듭니다

```
prompt2 = "목표: admin 계정이 로그인에 몇 번 실패했는지 알려 주세요. "
prompt2 = prompt2 + "도구 count_failed_logins 를 실행한 결과는 4 입니다. "
prompt2 = prompt2 + "이 결과로 목표에 한 문장으로 답하세요."
print(call_llm(prompt2))
```

```
admin 계정의 로그인 실패 횟수는 총 4번입니다.
```

결과는 4 입니다	**우리 도구가 센 값을 프롬프트에 넣음**
str(result)	**숫자를 문자열에 이어 붙일 때 문자열로 바꿈 (9/22)**
call_llm	**고르는 것이 아니라 문장을 만드는 호출. choose_tool 이 아님**

5교시에 배운 것을 정리합니다

하려는 일	쓰는 코드
도구를 만든다	`def count_failed_logins(args): … return count`
LLM 에게 고르게 한다	`call_llm(tools_text + "목표: " + goal)`
고른 것을 읽는다	`choice = parse_llm_json(text) · choice["tool"] · choice["args"]`
위 두 줄을 한 번에	`choice = choose_tool(goal)`
우리 코드로 실행한다	`if choice["tool"] == "…": result = 도구(choice["args"])`
결과로 답을 만든다	`call_llm("목표: … 실행한 결과는 … 한 문장으로 답하세요.")`

실습 ④ · 노트북 5교시 절 · 261006_pm_agent_tools.ipynb

문제 5-1 ~ 5-5 · 도전 ⭐5-6

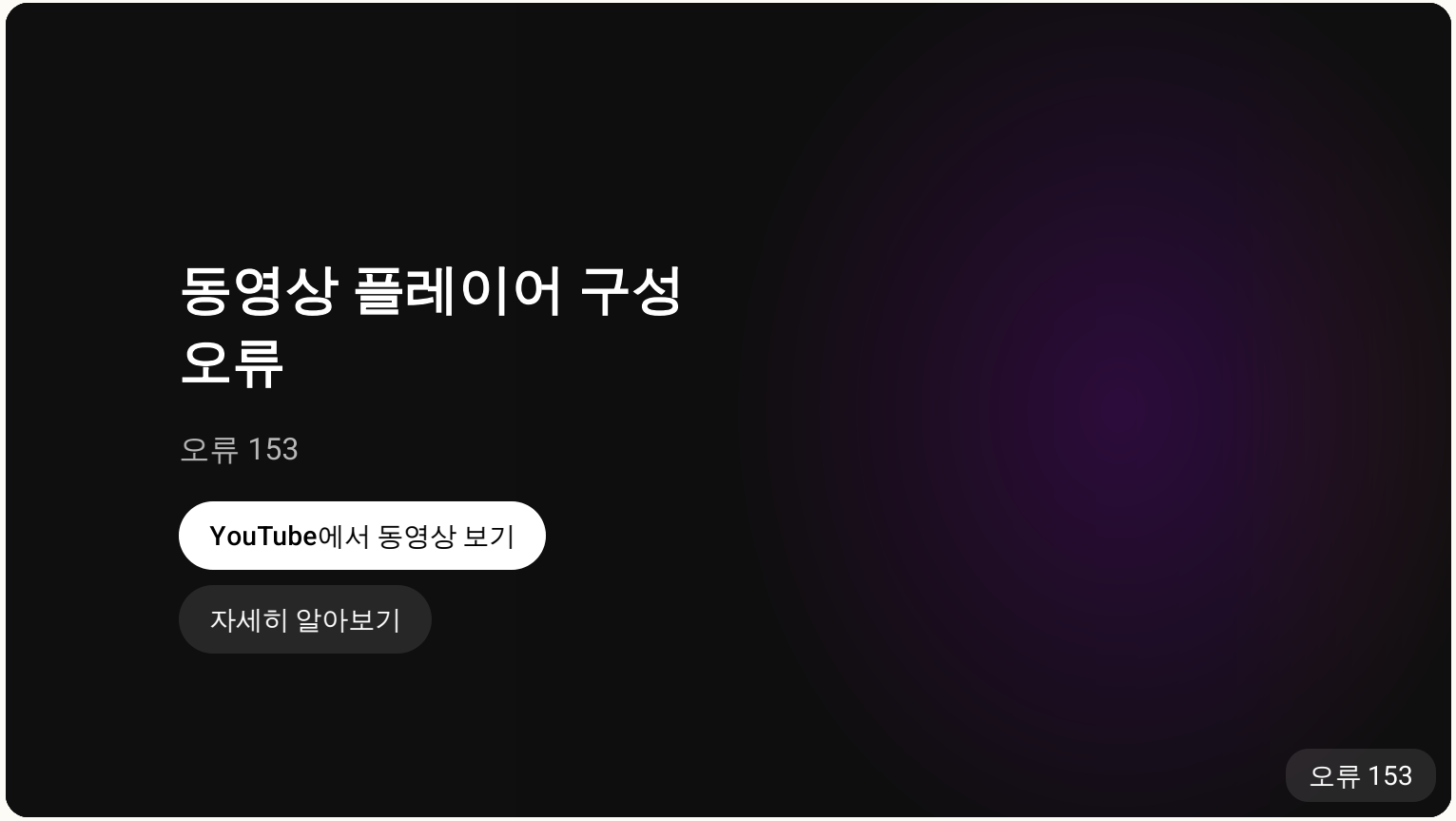
① **5교시 맨 위 준비 셀 넷을 먼저 실행합니다**
· 오전을 못 끝냈어도 여기서 시작할 수 있습니다

② **문제 5-1 ~ 5-2 — 도구 만들기**
· LLM 을 호출하지 않습니다. 5.3 아래 준비 셀도 실행합니다

③ **문제 5-3 ~ 5-5 — 고르기 · 실행 · 답**
· 429 면 1분 뒤 다시 실행. ⭐5-6 은 LLM 을 두 번 호출

영상으로 매개변수와 인자를 봅니다 (3분 22초)

함수를 부를 때 넣은 값(인자)이 함수의 자리(매개변수)로 들어가 실행됩니다.



6교시에 사용하는 용어는 두 개입니다

용어 (읽는 법)	뜻	6교시의 예
레지스트리 (registry)	이름과 함수를 짝지어 둔 딕셔너리	`tool_registry = {"lookup_ip": lookup_ip}`
라우터 (router)	이름을 보고 실제 함수를 찾아 실행하는 코드	`route_tool_call(choice)`

레지스트리는 도구 이름과 **실제 함수**를 짝지어 둔 딕셔너리입니다

> LLM 이 주는 것은 도구 이름(글자)뿐입니다. 그 글자로 실행할 함수를 찾으려면 짝지어 둔 딕셔너리가 필요합니다.

	9/23 에 쓴 딕셔너리	레지스트리
모양	`{"kim": 3, "lee": 1}`	`{"lookup_ip": lookup_ip, …}`
키	`"kim"` — 계정 이름	`"lookup_ip"` — LLM 이 답하는 도구 이름
값	`3` — 숫자	`lookup_ip` — 함수(괄호 없이)
키로 꺼내면	실패 횟수 `3`	실행할 함수

라우터는 LLM 이 고른 이름으로 레지스트리에서 함수를 찾아 실행합니다

레지스트리에 `count_failed_logins` 와 `lookup_ip` 두 도구가 있습니다.
LLM 이 고른 이름 두 개를 따라갑니다.

단계	"lookup_ip" 를 받으면	"delete_all_logs" 를 받으면
① 이름을 꺼냄	"lookup_ip"	"delete_all_logs"
② 레지스트리에서 찾음	있음 → lookup_ip 함수	없음
③ 결과	함수를 실행해 결과를 돌려줌	[오류] 등록되지 않은 도구: delete_all_logs

5교시 `if · elif` 가 하던 일과 같습니다.
도구가 늘어도 라우터는 그대로이고, 없는 이름은 알려 줍니다.

레지스트리는 도구 목록이고
라우터는 그 목록을 보고 실행하는 함수입니다

LLM
도구 이름을 고름

→

라우터
그 이름을
레지스트리에서
찾음

→

레지스트리
이름에 맞는
함수를 줌

→

라우터
그 함수를 실행

	레지스트리 tool_registry	라우터 route_tool_call
정체	딕셔너리	함수
담긴 것	도구 이름 → 함수 짝	찾고 · 실행하고 · 없으면 알리는 순서
혼자서는	적혀 있기만 하고 아무것도 안 함	찾을 곳이 없어 아무것도 못 함
도구가 늘면	한 줄 더함	고치지 않음

6교시에는 네 단계로 `tool_router.py` 를 만듭니다

만들 것은 두 가지 — 도구 목록(레지스트리)과, 그 목록을 보고 실행하는 함수(라우터)입니다.

단계	하는 일	다 되면
① 도구 목록 만들기 레지스트리 · 문제 6-2	도구 두 개를 이름 → 함수로 딕셔너리에 적음	도구 2개가 등록됨
② 실행 함수 만들기 라우터 · 문제 6-3	이름을 받으면 ①에서 찾아 실행, 없으면 알림	찾으면 결과 · 없으면 [오류]
③ 파일로 저장 문제 6-4	①②를 `tool_router.py` 한 파일에 담음	파일이 생김
④ LLM 과 연결 문제 6-5	LLM 이 고른 이름을 ②에 넘김	park.js 의 실패 횟수 1 이 나옴

LLM 이 목록에 없는 도구 이름을 답하면

`if` · `elif` 갈래는 아무것도 하지 않고 지나칩니다

	5교시 — if · elif 갈래	6교시 — 레지스트리 + 라우터
도구가 열 개면	갈래도 열 개	딕셔너리에 한 줄씩 더함
LLM 이 `delete_all_logs` 를 답하면	어느 갈래에도 안 걸리고 지나감	[오류] 등록되지 않은 도구 로 알림
왜 아무 일도 없었나	아무도 모름	화면에 남아 바로 앎
실행하는 코드	도구를 더할 때마다 고침	고치지 않음

그래서 6교시에는 이름 → 함수 딕셔너리(레지스트리)와 찾아 실행하는 코드(라우터)를 만듭니다.

레지스트리에는 함수를 괄호 없이 담습니다

레지스트리는 함수를 지금 실행하는 곳이 아니라, LLM 이 도구를 고른 뒤에 실행할 함수를 미리 보관해 두는 곳입니다.
그래서 담을 때는 괄호 없이 함수 이름만 씁니다.

```
def say_ok(args):
    return "ok " + args["name"]
good = {"ok": say_ok}                       # 괄호 없음
bad  = {"ok": say_ok({"name": "deploy"})}   # 괄호 있음
```

	딕셔너리에 담긴 것	나중에 꺼내 실행하면
good (괄호 없음)	함수 say_ok	"ok deploy" — 실행됨
bad (괄호 있음)	이미 실행한 결과 "ok deploy" (글자)	TypeError — 글자는 실행할 수 없음

함수를 딕셔너리에 담고 이름으로 꺼내 실행합니다

```
registry = {"ok": say_ok}          # 괄호가 없다 - 담기만 한다
func = registry.get("ok")           # 이름으로 함수를 꺼낸다
print(func({"name": "deploy"}))     # 꺼낸 함수를 실행한다
print(registry.get("없는이름"))
```

```
ok deploy
None
```

`registry.get("ok")`	키로 값을 꺼냄 (9/22). 여기서 값은 함수
`.get("없는이름")`	키가 없으면 오류 없이 None 을 반환함
도구가 늘면	딕셔너리에 한 줄만 더함. 꺼내 실행하는 코드는 그대로

레지스트리의 키는 LLM 에게 알려 준 이름과 같아야 합니다

```
tool_registry = {"count_failed_logins": count_failed_logins,
                 "lookup_ip": lookup_ip}
print(len(tool_registry))
```

```
2
```

키 "count_failed_logins"	tools_text 에 적은 이름. 한 글자도 다르면 못 찾음
값 count_failed_logins	괄호 없는 함수 이름 — 함수 자체
len(tool_registry)	등록한 도구의 수

등록되지 않은 이름은 무시하지 않고 오류로 알립니다

```
func = tool_registry.get("delete_all_logs")    # None
if func:
    result = func(args)
else:
    print("[오류] 등록되지 않은 도구: delete_all_logs")
```

```
[오류] 등록되지 않은 도구: delete_all_logs
```

`if func:`	함수를 찾았는지 확인함. None 이면 실행하지 않음
`print("[오류] …")`	조용히 넘어가지 않고 화면에 남김

라우터는 찾으면 실행하고 없으면 None 을 반환합니다

```
def route_tool_call(choice):
    name = choice["tool"]
    func = tool_registry.get(name)
    if func:
        return func(choice["args"])
    print("[오류] 등록되지 않은 도구:", name)
    return None
print(route_tool_call({"tool": "count_failed_logins",
                       "args": {"user": "park.js"}}))
print(route_tool_call({"tool": "block_everything", "args": {}}))
```

```
1
[오류] 등록되지 않은 도구: block_everything
None
```

라우터에 else 가 없는 이유는 return 에서 함수가 끝나기 때문입니다

그래서 오류 두 줄은 if 밖에 둡니다. 함수를 찾으면 그 두 줄은 실행되지 않습니다.

	함수를 찾았을 때	못 찾았을 때
if func:	참 → 안으로 들어감	거짓 → 건너뜀
return func(…)	여기서 함수가 끝남	실행되지 않음
print("[오류] …")	실행되지 않음	이 줄부터 실행됨
return None	실행되지 않음	None 을 반환함

tool_router.py 에 도구 둘 · 레지스트리 · 라우터를 담습니다

오후의 산출물입니다. 문제 6-2 와 6-3 에서 쓴 줄을 파일 하나에 다시 씁니다.

파일 안의 순서	하는 일
① `import json` · `import requests`	도구가 쓰는 모듈을 불러옴
② `count_failed_logins` · `lookup_ip`	도구 두 개를 정의함
③ `tool_registry`	도구를 등록함. 함수를 정의한 뒤에 만들어야 담을 수 있음
④ `route_tool_call`	이름으로 함수를 찾아 실행함
확인 셀	`park.js` 를 세어 `1` 이 나오면 성공

LLM 이 고른 도구를 라우터가 한 줄로 실행합니다

```
goal = "admin 계정이 로그인에 몇 번 실패했는지 알려 주세요."
choice = choose_tool(goal)
if choice:
    result = route_tool_call(choice)
    print(choice["tool"], "→", result)

count_failed_logins → 4
```

`if choice:`	**LLM 의 답을 읽지 못했으면 None — 확인한 뒤에 넘김**
`route_tool_call(choice)`	**5교시의 `if·elif` 갈래가 한 줄로 바뀜**

LLM 이 못 고르면 아무 도구도 실행되지 않습니다

```
goal = "오늘 점심 메뉴를 추천해 주세요."
choice = choose_tool(goal)
if choice:
    print(route_tool_call(choice))
else:
    print("도구를 고르지 못했습니다")
```

단계	LLM 이 문장으로 답하면	LLM 이 null 로 답하면
① LLM 의 답	「관련된 도구가 없습니다.」	JSON {"tool": null, "args": {}}
② choice	None (못 읽음)	딕셔너리 (읽힘)
③ if choice:	거짓 → else 로	참 → 라우터로
④ 화면	도구를 고르지 못했습니다	[오류] 등록되지 않은 도구: None

6교시에 배운 것을 정리합니다

하려는 일	쓰는 코드
도구를 등록한다	`tool_registry["lookup_ip"] = lookup_ip` **(괄호 없이)**
이름으로 함수를 꺼낸다	`func = tool_registry.get(name)`
꺼낸 함수를 실행한다	`func(choice["args"])`
없는 이름을 알린다	`print("[오류] 등록되지 않은 도구:", name)`
한 줄로 실행한다	`route_tool_call(choice)`

실습 ⑤ · 노트북 6교시 절 · 261006_pm_agent_tools.ipynb

문제 6-1 ~ 6-5 · 선택 6-6 · 도전 ⭐6-7

① **6교시 준비 셀을 먼저 실행합니다**
· 5교시를 못 끝냈어도 여기서 시작할 수 있습니다

② **문제 6-1 ~ 6-4 — 레지스트리 · 라우터 · tool_router.py**
· LLM 을 호출하지 않습니다

③ **문제 6-5 — LLM 이 고르고 라우터가 실행**
· 429 면 1분 뒤 다시 실행

7교시에 사용하는 용어는 세 개입니다

용어 (읽는 법)	뜻	7교시의 예
승인 게이트 (approval gate)	위험한 실행 앞에 **사람 확인**을 두는 것	`if choice["tool"] in need_approval:`
사람 개입 (human-in-the-loop · 휴먼 인 더 루프)	자동 흐름 중간에 사람이 끼는 설계	담당자의 답 `answer = "y"`
held (헬드 · 보류됨)	승인을 받지 못해 실행하지 않고 멈춰 둔 상태를 나타내는 문자열	`return "held"`

7교시에는 승인 게이트와 기록을 만듭니다

위험한 도구는 사람이 승인해야 실행하고, 못 받으면 `held`(보류됨)로 남깁니다.

고르기 **choose_tool** → **승인 게이트** 필요하면 사람 확인 → 실행 **route_tool_call** → 기록 **agent_result.json**

순서	만드는 것	노트북	끝나면 보이는 값
① 잠금 도구	계정 잠금 `lock_account` (출력만)	문제 7-2	`[조치] 계정 잠금: guest01`
② 게이트	답이 "y" 일 때만 실행하는 `run_with_gate`	문제 7-3	`1 · held · locked`
③ 기록	한 일을 파일로 남김	문제 7-4	`agent_result.json` 이 생김
④ 전체 흐름	고르기부터 기록까지 한 번에	문제 7-5	`'result': 'locked'`

계정 잠금처럼 되돌리기 어려운 도구 앞에는 사람의 확인을 둡니다

> 5교시에 봤듯이 LLM 은 엉뚱한 도구를 고를 수 있습니다.

	바로 실행해도 되는 도구	승인을 받아야 하는 도구
도구	`count_failed_logins` · `lookup_ip`	`lock_account`
틀렸을 때	다시 하면 됨	그 사람이 일을 못 하고, 되돌리는 데 사람 손이 듦
실행	LLM 이 고르면 바로 실행	담당자의 답이 "y" 일 때만 실행

승인이 필요한 도구는 코드의 리스트에 정해 둡니다

```
need_approval = ["lock_account"]
print("lock_account" in need_approval)
print("lookup_ip" in need_approval)
```

```
True
False
```

`"lock_account" in need_approval`	리스트 안에 그 값이 있으면 **True (9/23)**
규칙의 자리	승인 여부를 **LLM 이 정하지 않음**. 코드에 적혀 있어야 언제나 같게 동작함
오늘의 `lock_account`	실습이라 화면에 출력만 함. 실제 계정을 잠그지 않음

lock_account 는 출력만 하고 locked 를 반환합니다

계정 잠금 도구입니다. 실습이라 실제로 잠그지 않습니다.

```
def lock_account(args):
    print("[조치] 계정 잠금:", args["user"])
    return "locked"
tool_registry["lock_account"] = lock_account
print(route_tool_call({"tool": "lock_account",
                       "args": {"user": "ops.kim"}}))
```

```
[조치] 계정 잠금: ops.kim
locked
```

print("[조치] …")	실제로 잠그는 대신 화면에 출력만 함
return "locked"	도구의 결과. 나중에 기록에 남김

승인이 필요한 도구는 답이 "y" 일 때만 실행합니다

```
answer = "n"            # 담당자의 답
if answer == "y":
    print("실행합니다")
else:
    print("보류합니다")
```

```
보류합니다
```

answer	담당자의 답. 실습에서는 변수에 직접 적음
보류	실행하지 않고 "held"(보류됨)라는 문자열로 남김
실제 서비스	답을 승인 화면이나 메신저에서 받음. 5과목에서 다룸

"y" 일 때만 실행합니다.

"y" 가 아닌 모든 답은 실행하지 않습니다.

담당자의 답 answer	answer == "y"	결과
"y"	True	실행
"n"	False	보류 — "held"
"Y" (대문자)	False	보류 — 대문자는 다른 글자
"yes"	False	보류
"" (빈 문자열)	False	보류

승인 게이트는 승인이 필요한 도구에만 답을 확인합니다

choice
LLM 이 고른 도구 → need_approval 안?
아니면 바로 실행 → answer == "y"?
아니면 held → route_tool_call
실행

부른 것	답	반환된 것
count_failed_logins · lee.yh	"n"	1 — 승인이 필요 없어 그대로 실행
lock_account · lee.yh	"n"	held — 보류
lock_account · lee.yh	"y"	[조치] 계정 잠금: lee.yh · locked

`run_with_gate` 는 y 가 아니면 held 를 반환하고 끝납니다

```
need_approval = ["lock_account"]
def run_with_gate(choice, answer):
    if choice["tool"] in need_approval:
        if answer != "y":
            return "held"
    return route_tool_call(choice)
```

!= (같지 않다)	"y" 가 아니면 참
`return "held"`	「보류됨」을 반환하고 그 자리에서 함수가 끝남. 도구는 실행되지 않음
`return route_tool_call(choice)`	승인이 필요 없거나 답이 "y" 일 때만 이 줄에 옴

에이전트가 한 일은 목표 · 도구 · 결과로 남깁니다

오후 산출물은 agent_result.json 입니다.

```
records = []
records.append({"goal": "admin 계정이 몇 번 실패했나",
                "tool": "count_failed_logins", "result": 4})
with open("agent_result.json", "w", encoding="utf-8") as f:
    json.dump(records, f, ensure_ascii=False, indent=2)
```

records.append({…})	기록 한 건은 딕셔너리. **goal · tool · result**
json.dump	리스트를 JSON 파일로 저장함 (9/28 오후)
ensure_ascii=False · indent=2	한글을 그대로, 두 칸 들여쓰기로 저장함

보류한 조치도 held 로 기록에 남깁니다

남기지 않으면 「왜 조치가 안 됐는지」 를 설명할 수 없습니다.

> 목표 세 개를 처리하고 담당자의 답이 "n" 이었을 때 남는 기록입니다.

goal	tool	result
admin 계정이 로그인에 몇 번 실패했는지	count_failed_logins	4
211.45.12.9 는 어느 나라의 주소인지	lookup_ip	South Korea / SamsungSDS Inc.
admin 계정이 공격받고 있습니다 …	lock_account	held

목표 하나를 고르기 → 게이트 → 실행 → 기록합니다

```
goal = ("admin 계정이 공격받고 있습니다. "
        "더 이상 로그인하지 못하게 해 주세요.")
answer = "y"
choice = choose_tool(goal)
if choice:
    result = run_with_gate(choice, answer)
    records.append({"goal": goal, "tool": choice["tool"],
                    "result": result})
```

```
[조치] 계정 잠금: admin
```

run_with_gate	라우터 대신 게이트에 넘김. 답이 "y" 라 실행
기록	{'tool': 'lock_account', 'result': 'locked'} — 답이 "n" 이면 held

7교시에 배운 것을 정리합니다

판단은 LLM 이, 실행은 우리 코드가, 되돌리기 어려운 결정은 사람이 합니다.

하려는 일	쓰는 코드
승인이 필요한 도구를 정한다	`need_approval = ["lock_account"]`
승인이 필요한지 확인한다	`if choice["tool"] in need_approval:`
승인하지 않으면 보류한다	`if answer != "y": return "held"`
한 일을 기록한다	`records.append({"goal": …, "tool": …, "result": …})`
파일로 남긴다	`json.dump(records, f, ensure_ascii=False, indent=2)`

실습 ⑥ · 노트북 7교시 절 · 261006_pm_agent_tools.ipynb

문제 7-1 ~ 7-5 · 도전 ⭐7-6

① **7교시 준비 셀을 먼저 실행합니다**
· ModuleNotFoundError 면 맨 아래 「정답 6-4」 셀을 실행

② **문제 7-1 ~ 7-4 — 승인 목록 · 잠금 도구 · 게이트 · 기록**
· LLM 을 호출하지 않습니다

③ **문제 7-5 — 고르기부터 기록까지**
· 429 면 1분 뒤 다시 실행. ⭐7-6 은 LLM 을 세 번 호출

8교시 — 오늘 만든 파일 셋이 있는지 확인합니다

새로 배우는 것은 없습니다. 없음 이 나오면 그 파일을 만드는 문제로 돌아갑니다.

노트북 8교시의 확인 셀을 실행합니다. 있음 : 이 세 줄 나오면 됩니다.

파일	만든 곳	하는 일
llm_client.py	오전 4교시	LLM 에 질문을 보내고, 답을 딕셔너리로 읽음
tool_router.py	오후 문제 6-4	도구를 등록하고, 이름으로 찾아 실행함
agent_result.json	오후 문제 7-4 · ⭐7-6	에이전트가 한 일의 기록. 보류한 것도 남음

실습 ⑦ · 8교시 남은 시간

ALEPH(알레프) 기본 과제 12개 풀기

각 과제에 투입할 수 있는 시간을 정하고, 그 안에 끝나도록 AI 를 활용합니다.
앞 과제가 승인돼야 다음 과제가 열립니다. 자기 차례의 과제부터 합니다.

과제	제목	계획 시간	과제	제목	계획 시간
T01	나를 소개하는 한 페이지	4~5h	T07	플랜두씨 다이어리 2 — 인증	6~8h
T02	내가 설계한 미니게임	6~7h	T08	내 소개 페이지에 패스키 달기	6~7h
T03	짤·카드 스튜디오	4~5h	T09	자동화 관제소	5~6h
T04	오늘의 진짜 정보판	6~7h	T10	AI와 함께 쓰는 첫 논문	8~10h
T05	대화가 끊겨도 이어지는 프로젝트	4~5h	T11	3만자 장편소설	6~8h
T06	플랜두씨 다이어리 1	8~10h	BR-A / B	마지막 과제 — 둘 중 하나	4~7h

내일은 이벤트 수십 건을 나눠 요약해 일일 보고서를 만듭니다

	오늘	내일
다루는 양	로그 한 줄 · 목표 하나씩	이벤트 수십 건을 나눠서
만드는 것	`llm_client.py` · `tool_router.py` · `agent_result.json`	일일 보고서
쓰는 것	—	오늘 만든 `llm_client.py` 그대로

🔔 **17:30 — 수업을 마치고 오후 리추얼로 넘어갑니다. 리추얼은 20분이고 17:50 에 끝납니다.**